

Developer Brief — happileeverafter2027.com

Project: Wedding website for Gillian & Merrick Lee **Domain:** happileeverafter2027.com

Wedding Date: August 1, 2027 — Jamaica **Hashtag:** #HappiLeeEverAfter2027 **Theme:** Old-world royal · Fairy tale · Greek/Aegean Renaissance · Caribbean accents **Reference Mockup:** [mockup_2_aegean_renaissance_v12.html](#) (provided)

Table of Contents

1. [Project Overview](#)
 2. [Tech Stack & Setup](#)
 3. [Design System](#)
 4. [Project Structure](#)
 5. [Page Sections \(in order\)](#)
 6. [Component Architecture](#)
 7. [JSX Implementation](#)
 8. [Amplitude Analytics Integration](#)
 9. [RSVP Backend & Email Capture](#)
 10. [Responsive Behavior](#)
 11. [Accessibility Requirements](#)
 12. [Performance & SEO](#)
 13. [Deployment Checklist](#)
 14. [Future Phases](#)
-

1. Project Overview

A single-page wedding website celebrating Gillian & Merrick Lee's engagement (Santorini, 2026) and upcoming wedding (Jamaica, August 1, 2027). The site collects RSVPs, captures emails for future event announcements, displays a photo/video gallery, and lets guests leave well-wishes in a guestbook.

Goals

- **Primary:** Capture RSVPs for multiple events (engagement soirée, bridal shower, wedding) with persistent guest details

- **Secondary:** Build email list for event announcements
- **Tertiary:** Provide a beautiful, share-worthy destination guests will reference repeatedly

Brand Personality

Old-world royal meets fairy-tale romance with Caribbean warmth. Think illuminated manuscripts, family crests, Greek inscriptions — paired with warm parchment tones and emerald accents that nod to Jamaica.

2. Tech Stack & Setup

Recommended Stack

Layer	Choice	Why
Framework	Next.js 14+ (App Router)	SSG/ISR, built-in image optimization, easy Vercel deploy
Language	TypeScript	Type safety across components, props, and event tracking schemas
Styling	Tailwind CSS + CSS variables	Utility classes for layout, CSS vars for design tokens
Animation	Framer Motion	Smooth fade-in animations on scroll/load
Forms	react-hook-form + zod	Validation for the RSVP modal
Analytics	@amplitude/analytics-browser	Per the spec
Email Backend	Resend + Vercel API routes	Lightweight, modern alternative to SendGrid
RSVP Storage	Airtable OR Supabase	Airtable for non-technical Merrick/Gillian to view; Supabase if more control needed
Hosting	Vercel	Native Next.js support, automatic deployments from Git
Domain	<code>happileeeeverafter2027.com</code>	Already owned by client

Required Environment Variables

Create `.env.local`:

```
bash
```

```
NEXT_PUBLIC_AMPLITUDE_API_KEY=a013a8aef5661df201bf8330107f5244
NEXT_PUBLIC_WEDDING_DATE=2027-08-01T16:00:00
RESEND_API_KEY=<obtain from resend.com>
AIRTABLE_API_KEY=<obtain from airtable.com>
AIRTABLE_BASE_ID=<your base id>
AIRTABLE_RSVP_TABLE=RSVPs
AIRTABLE_EMAIL_TABLE=EmailSubscribers
NOTIFICATION_EMAIL=merrick@example.com # where RSVP notifications are sent
```

Initial Setup Commands

```
bash
```

```
npx create-next-app@latest happilee --typescript --tailwind --app --eslint
cd happilee
npm install @amplitude/analytics-browser framer-motion react-hook-form zod resend
npm install -D @types/node
```

3. Design System

3.1 Color Tokens

Add to `app/globals.css`:

```
css
```

```
:root {  
  /* Backgrounds (parchment palette) */  
  --color-bg: #f7f1e3;  
  --color-bg-warm: #efe5cc;  
  --color-bg-cream: #faf6ec;  
  
  /* Navy (primary text & headings) */  
  --color-navy: #1a3a5c;  
  --color-navy-deep: #0f2740;  
  --color-navy-light: #2c5282;  
  
  /* Gold (accents, borders, ornaments) */  
  --color-gold: #b8893b;  
  --color-gold-bright: #d4a548;  
  
  /* Rose (Santorini accents, ampersands, eyebrows) */  
  --color-rose: #a4243b;  
  --color-rose-deep: #7a1a2c;  
  --color-rose-soft: #c44560;  
  
  /* Emerald (Jamaica accents) */  
  --color-emerald: #1a5e3a;  
  --color-emerald-deep: #0f3d24;  
  --color-emerald-bright: #2d8050;  
  
  /* Ink (body text) */  
  --color-ink: #2a2a2a;  
}
```

Extend `tailwind.config.ts`:

ts

```

theme: {
  extend: {
    colors: {
      bg: 'var(--color-bg)',
      'bg-warm': 'var(--color-bg-warm)',
      'bg-cream': 'var(--color-bg-cream)',
      navy: { DEFAULT: 'var(--color-navy)', deep: 'var(--color-navy-deep)', light: 'v
      gold: { DEFAULT: 'var(--color-gold)', bright: 'var(--color-gold-bright)' },
      rose: { DEFAULT: 'var(--color-rose)', deep: 'var(--color-rose-deep)', soft: 'v
      emerald: { DEFAULT: 'var(--color-emerald)', deep: 'var(--color-emerald-deep)',
      ink: 'var(--color-ink)',
    }
  }
}

```

3.2 Color Usage Rules

- **Backgrounds:** alternate between `bg`, `bg-warm`, `bg-cream`, and `navy-deep` (signup/footer) for visual rhythm
- **Headings:** `navy-deep`
- **Body text:** `ink`
- **Eyebrows/labels (small caps text above titles):** `gold`
- **Italic accent text (subtitles, signatures):** `gold` or `gold-bright`
- **Ampersands & section eyebrows:** `rose`
- **Jamaica-specific elements (hero location, wedding event card, "Crowned in Jamaica" text):** `emerald`
- **Borders:** `gold` at full or 30-40% opacity; `emerald` for Jamaica-themed cards

3.3 Typography

Load via `next/font/google`:

tsx

```
// app/fonts.ts
import { Italiana, Playfair_Display, Tenor_Sans } from 'next/font/google'

export const italiana = Italiana({
  weight: '400',
  subsets: ['latin'],
  variable: '--font-italiana',
})

export const playfair = Playfair_Display({
  subsets: ['latin'],
  style: ['normal', 'italic'],
  weight: ['400', '500', '600', '700'],
  variable: '--font-playfair',
})

export const tenor = Tenor_Sans({
  weight: '400',
  subsets: ['latin'],
  variable: '--font-tenor',
})
```

Font roles:

Font	Usage
Italiana	Hero names, section titles, event titles, count numbers, "Lee" footer wordmark
Playfair Display (italic)	Hero script, subtitles, drop caps, signatures, body emphasis
Playfair Display (regular)	Body paragraphs
Tenor Sans	All eyebrows, labels, button text, small caps elements (everything with letter-spacing)

3.4 Spacing & Layout Constants

CSS

```

/* Section padding scale */
--section-pad-desktop: 7rem 2rem;
--section-pad-tablet: 6rem 2rem;
--section-pad-mobile: 4.5rem 1.5rem;
--section-pad-small: 3.5rem 1.25rem;

/* Container max-width */
--container-max: 1200px;

/* Section header bottom margin */
--header-margin: 5rem (desktop), 4rem (tablet), 3rem (mobile);

```

3.5 The Lee Crest

- File: `Lee_Crest_2026.png` (provided)
- Featured prominently in **hero** and **footer**
- Sizing:
 - Hero desktop: ~260px wide
 - Hero mobile: 240px
 - Hero small mobile: 200px
 - Footer: 140px (desktop), 130px (mobile)
- Apply `drop-shadow(0 8px 24px rgba(15, 39, 64, 0.25))` in hero for lift
- Use Next.js `<Image>` with `priority` flag in hero

3.6 Greek Key Border (Decorative)

Repeating SVG pattern at top and bottom of hero, between guestbook and footer. Color: `var(--color-gold)` at 60% opacity. Height 24px, repeats horizontally every 40px. SVG data URI:

```

css

.greek-key {
  height: 24px;
  background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg
  background-repeat: repeat-x;
  background-size: 40px 24px;
  opacity: 0.6;
}

```

4. Project Structure

```
happilee/
├─ app/
│   ├─ layout.tsx           # Root layout, fonts, Amplitude provider
│   ├─ page.tsx             # Single-page composition
│   ├─ globals.css          # CSS variables, base styles, Greek key
│   ├─ fonts.ts             # Google Fonts configuration
│   ├─ api/
│   │   ├─ rsvp/route.ts    # POST endpoint for RSVPs
│   │   └─ subscribe/route.ts # POST endpoint for email signup
│   └─ components/
│       ├─ sections/
│       │   ├─ Hero.tsx
│       │   ├─ Countdown.tsx
│       │   ├─ OurStory.tsx
│       │   ├─ Gallery.tsx
│       │   ├─ Events.tsx
│       │   ├─ EmailSignup.tsx
│       │   ├─ Guestbook.tsx
│       │   └─ Footer.tsx
│       └─ ui/
│           ├─ SectionHeader.tsx
│           ├─ Flourish.tsx
│           ├─ GreekKey.tsx
│           └─ Button.tsx
│       └─ modals/
│           └─ RsvpModal.tsx
├─ lib/
│   ├─ amplitude.ts         # Amplitude initialization & event helpers
│   ├─ storage.ts           # localStorage helpers for guest info
│   └─ api.ts               # Fetch wrappers for RSVP/email endpoints
├─ public/
│   ├─ assets/
│   │   ├─ Lee_Crest_2026.png
│   │   ├─ proposal-hero.jpg # Santorini engagement photo
│   │   └─ gallery/         # All gallery images & videos
│   ├─ favicon.ico
│   └─ og-image.jpg         # Open Graph share image
├─ types/
│   └─ index.ts             # TypeScript interfaces
├─ .env.local
└─ next.config.ts
```



```
└─ tailwind.config.ts
└─ package.json
```

5. Page Sections (in order)

#	Section	Component	Background	Notes
1	Greek Key (top)	<code><GreekKey /></code>	bg	Decorative border
2	Hero	<code><Hero /></code>	bg with gradient	Crest, "Gillian & Merrick", date, Jamaica location
3	Greek Key (bottom of hero)	<code><GreekKey /></code>	bg	Decorative border
4	Countdown	<code><Countdown /></code>	bg → bg-warm gradient	Live timer to wedding date
5	Our Story	<code><OurStory /></code>	bg	Two-column: image + narrative
6	Gallery	<code><Gallery /></code>	bg-warm	Mosaic of photos/videos
7	Upcoming Celebrations	<code><Events /></code>	bg	Three event cards with RSVP buttons
8	Email Signup	<code><EmailSignup /></code>	navy-deep	Newsletter capture
9	Guestbook	<code><Guestbook /></code>	bg	Public well-wishes
10	Greek Key	<code><GreekKey /></code>	navy-deep	Transition to footer
11	Footer	<code><Footer /></code>	navy-deep	Crest, tagline, hashtag
-	RSVP Modal	<code><RsvpModal /></code>	(overlay)	Triggered from event cards

Note: Registry section is hidden for now (will be added in a future phase). Keep the component built but don't render it.

5.1 Section-by-Section Content Spec

Hero

- **Crest** (image, top-centered)
- **Eyeblink:** OFFICIALLY ENGAGED · SEALED IN SANTORINI
- **Script:** *"It is with hearts full of joy that we announce"*
- **Names:** Gillian & Merrick (ampersand in rose)
- **Date:** SUMMER · 2027
- **Location:** ~ The Isle of Jamaica ~ (in emerald)

Countdown

- **Eyeblink:** A Joyful Anticipation
- **Title:** Counting the Days
- Live ticker showing Days (3 digits, zero-padded), Hours, Minutes, Seconds (2 digits each)
- Update every 1000ms; clear interval when wedding date passes

Our Story

- **Eyeblink:** Chapter One
- **Title:** Our Love Story
- **Subtitle:** *Two souls, one beautiful chapter beginning beneath the Aegean sun*
- Left column: proposal image with corner ornaments
- Right column: chapter label, sub-heading, paragraphs with drop cap, signature ~ With all our love, M & G ~

Gallery

- **Eyeblink:** ~ A Curated Album ~
- **Title:** The Gallery
- 4-column mosaic (desktop), 2-column (mobile), 1-column (small mobile)
- Mix of large (2x2), tall (1x2), wide (2x1), and standard cells
- Video items show a circular play icon overlay

Events

- **Eyeblink:** Save These Dates
- **Title:** Upcoming Celebrations
- **Subtitle:** *A series of joyful gatherings leading to our forever · Dates to be announced*

- Three cards:
 1. **Engagement Soirée** — New York
 2. **Bridal Shower** — TBA
 3. **The Wedding** — Jamaica (featured with emerald styling)
- Each card has a "RSVP" button that opens the modal

Email Signup

- **Eye brow:** ~ You're Invited ~
- **Title:** Stay in the Know
- **Subtitle:** *Subscribe to receive announcements and event details as they unfold*
- Single email input + Subscribe button

Guestbook

- **Eye brow:** Leave Your Mark
- **Title:** Our Guestbook
- Name input + textarea for wishes + "Sign the Guestbook" button

Footer

- Lee crest (large)
 - Tagline: ~ Sealed in Santorini · Crowned in Jamaica ~ ("Crowned in Jamaica" in emerald)
 - Hashtag: #HappiLeeEverAfter2027
-

6. Component Architecture

6.1 Reusable UI Components

tsx

```
// components/ui/SectionHeader.tsx
interface SectionHeaderProps {
  eyebrow: string;
  title: string;
  subtitle?: string;
  showFlourish?: boolean;
}

// components/ui/Button.tsx
interface ButtonProps {
  variant: 'outline' | 'solid' | 'outline-emerald' | 'gold';
  children: React.ReactNode;
  onClick?: () => void;
  type?: 'button' | 'submit';
}

// components/ui/Flourish.tsx
// Renders the gold line + ornament + gold line decorative divider

// components/ui/GreekKey.tsx
// Renders the repeating Greek key border
```

6.2 Animation Pattern

Use Framer Motion for entrance animations. Standard pattern:

```
tsx

import { motion } from 'framer-motion'

<motion.div
  initial={{ opacity: 0, y: 20 }}
  whileInView={{ opacity: 1, y: 0 }}
  viewport={{ once: true, margin: '-100px' }}
  transition={{ duration: 1, delay: 0.3 }}
>
  {/* content */}
</motion.div>
```

For the hero, use staggered delays (0.3s, 0.6s, 0.9s, 1.2s, 1.5s, 1.8s) to fade elements in sequentially.

7. JSX Implementation

7.1 Root Layout

tsx

```
// app/layout.tsx
import type { Metadata } from 'next'
import { italiana, playfair, tenor } from './fonts'
import { AmplitudeProvider } from '@lib/amplitude'
import './globals.css'

export const metadata: Metadata = {
  title: 'Gillian & Merrick · #HappiLeeEverAfter2027',
  description: 'Sealed in Santorini · Crowned in Jamaica. Join us in celebrating the',
  openGraph: {
    title: 'Gillian & Merrick · Wedding 2027',
    description: '~ Sealed in Santorini · Crowned in Jamaica ~',
    images: ['/og-image.jpg'],
    type: 'website',
  },
  twitter: { card: 'summary_large_image' },
}

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en" className={` ${italiana.variable} ${playfair.variable} ${tenor.variable} `}>
      <body className="bg-bg text-ink font-playfair">
        <AmplitudeProvider>
          {children}
        </AmplitudeProvider>
      </body>
    </html>
  )
}
```

7.2 Page Composition

tsx

```
// app/page.tsx
import Hero from '@components/sections/Hero'
import Countdown from '@components/sections/Countdown'
import OurStory from '@components/sections/OurStory'
import Gallery from '@components/sections/Gallery'
import Events from '@components/sections/Events'
import EmailSignup from '@components/sections/EmailSignup'
import Guestbook from '@components/sections/Guestbook'
import Footer from '@components/sections/Footer'
import GreekKey from '@components/ui/GreekKey'
import RsvpModal from '@components/modals/RsvpModal'
import { RsvpProvider } from '@lib/rsvp-context'

export default function Page() {
  return (
    <RsvpProvider>
      <main>
        <Hero />
        <Countdown />
        <OurStory />
        <Gallery />
        <Events />
        <EmailSignup />
        <Guestbook />
        <GreekKey />
        <Footer />
      </main>
      <RsvpModal />
    </RsvpProvider>
  )
}
```

7.3 Hero Component

tsx

```
// components/sections/Hero.tsx
'use client'
import Image from 'next/image'
import { motion } from 'framer-motion'
import GreekKey from '@components/ui/GreekKey'

export default function Hero() {
  return (
    <section className="relative flex flex-col min-h-screen bg-bg overflow-hidden">
      <GreekKey />

      <div className="flex-1 flex items-center justify-center px-6 pt-12 pb-12 md:pt-12 md:pb-12">
        <div className="text-center max-w-4xl">

          <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            transition={{ duration: 1.2, delay: 0.2 }}
            className="mb-10"
          >
            <Image
              src="/assets/Lee_Crest_2026.png"
              alt="Lee Family Crest"
              width={520}
              height={520}
              priority
              className="mx-auto w-[200px] sm:w-[240px] md:w-[260px] h-auto"
              style={{ filter: 'drop-shadow(0 8px 24px rgba(15,39,64,0.25))' }}
            />
          </motion.div>

          <motion.div
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ duration: 1, delay: 0.5 }}
            className="font-tenor text-[0.65rem] sm:text-[0.7rem] tracking-[0.4em] s
          >
            Officially Engaged · Sealed in Santorini
          </motion.div>

          <motion.p
            initial={{ opacity: 0, y: 20 }}
            animate={{ opacity: 1, y: 0 }}

```

```

        transition={{ duration: 1, delay: 0.7 }}
        className="font-playfair italic text-xl md:text-3xl text-navy mb-4"
      >
        It is with hearts full of joy that we announce
      </motion.p>

      <motion.h1
        initial={{ opacity: 0, y: 20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 1.2, delay: 0.9 }}
        className="font-italiana text-6xl sm:text-7xl md:text-8xl lg:text-9xl text-emerald"
        style={{ letterSpacing: '0.03em' }}
      >
        Gillian
        <span className="block sm:inline mx-2 font-playfair italic text-rose text-2xl">
          Merrick
        </span>
      </motion.h1>

      <motion.div
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        transition={{ duration: 1, delay: 1.2 }}
        className="flex items-center justify-center gap-6 my-8"
      >
        <span className="h-px w-20 bg-gold" />
        <svg width="20" height="20" viewBox="0 0 20 20"><path d="M10,3 L12,8 L17,8" /></svg>
        <span className="h-px w-20 bg-gold" />
      </motion.div>

      <motion.div
        initial={{ opacity: 0, y: 20 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 1, delay: 1.4 }}
      >
        <p className="font-italiana text-2xl tracking-[0.2em] text-navy mb-2">SUGGESTION</p>
        <p className="font-playfair italic text-lg tracking-wide text-emerald">~
      </p>
    </motion.div>

  </div>
</div>

<GreekKey />
</section>

```



```
)  
}
```

7.4 Countdown Component

```
tsx
```

```
// components/sections/Countdown.tsx
'use client'
import { useEffect, useState } from 'react'
import SectionHeader from '@components/ui/SectionHeader'

const WEDDING_DATE = new Date(process.env.NEXT_PUBLIC_WEDDING_DATE!).getTime()

const pad = (n: number, w: number) => String(n).padStart(w, '0')

export default function Countdown() {
  const [time, setTime] = useState({ days: '000', hours: '00', minutes: '00', seconds: '00' })

  useEffect(() => {
    const tick = () => {
      const distance = WEDDING_DATE - Date.now()
      if (distance <= 0) {
        setTime({ days: '000', hours: '00', minutes: '00', seconds: '00' })
        return
      }
      const days = Math.floor(distance / 86400000)
      const hours = Math.floor((distance % 86400000) / 3600000)
      const minutes = Math.floor((distance % 3600000) / 60000)
      const seconds = Math.floor((distance % 60000) / 1000)
      setTime({ days: pad(days, 3), hours: pad(hours, 2), minutes: pad(minutes, 2), seconds: pad(seconds, 2) })
    }
    tick()
    const interval = setInterval(tick, 1000)
    return () => clearInterval(interval)
  }, [])

  const items = [
    { num: time.days, label: 'Days' },
    { num: time.hours, label: 'Hours' },
    { num: time.minutes, label: 'Minutes' },
    { num: time.seconds, label: 'Seconds' },
  ]

  return (
    <section className="py-28 px-6 bg-gradient-to-b from-bg to-bg-warm">
      <div className="max-w-6xl mx-auto">
        <SectionHeader eyebrow="A Joyful Anticipation" title="Counting the Days" show eyebrow="A Joyful Anticipation" />
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4 md:gap-8 max-w-4xl mx-auto">
          {items.map(item => (

```

```
      <div key={item.label} className="text-center py-10 px-4 bg-bg-cream border-2 border-emerald/20 pointer-events-none">
        <div className="absolute inset-1.5 border border-emerald/20 pointer-events-none">
          <div className="font-italiana text-5xl md:text-7xl text-navy-deep lead-10">
            <div className="font-tenor text-[0.7rem] tracking-[0.4em] text-rose-evaporate">
              {item.value}
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
)
}
```

7.5 RSVP Modal Component

tsx

```

// components/modals/RsvpModal.tsx
'use client'
import { useEffect, useState } from 'react'
import { useForm } from 'react-hook-form'
import { useRsvp } from '@/lib/rsvp-context'
import { trackEvent } from '@/lib/amplitude'
import { saveGuestInfo, loadGuestInfo, clearGuestInfo } from '@/lib/storage'

interface RsvpFormData {
  name: string
  email: string
  phone: string
  address: string
  guests: string
  attending: string
}

export default function RsvpModal() {
  const { isOpen, currentEvent, closeModal } = useRsvp()
  const [submitted, setSubmitted] = useState(false)
  const [savedInfo, setSavedInfo] = useState<Partial<RsvpFormData> | null>(null)
  const { register, handleSubmit, reset, setValue, formState: { errors, isSubmitting } } = useForm<RsvpFormData>()

  useEffect(() => {
    if (isOpen) {
      setSubmitted(false)
      const saved = loadGuestInfo()
      setSavedInfo(saved)
      if (saved) {
        Object.entries(saved).forEach(([key, value]) => {
          if (key !== 'attending') setValue(key as keyof RsvpFormData, value)
        })
      } else {
        reset()
      }
      trackEvent('rsvp_modal_opened', { event_name: currentEvent?.name, has_saved_info: !!saved })
    }
  }, [isOpen, currentEvent, setValue, reset])

  const onSubmit = async (data: RsvpFormData) => {
    trackEvent('rsvp_submit_attempted', {
      event_name: currentEvent?.name,
      attending: data.attending,
    })
    // ...submit logic
  }
}

```

```

    guest_count: data.guests,
  })

  try {
    const response = await fetch('/api/rsvp', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ ...data, event: currentEvent?.name }),
    })

    if (!response.ok) throw new Error('RSVP failed')

    saveGuestInfo({
      name: data.name,
      email: data.email,
      phone: data.phone,
      address: data.address,
      guests: data.guests,
    })

    trackEvent('rsvp_submit_success', { event_name: currentEvent?.name, attending:
    setSubmitted(true)
  } catch (error) {
    trackEvent('rsvp_submit_failed', { event_name: currentEvent?.name, error: Stri
    alert('Something went wrong. Please try again.')
  }
}

const handleClearSaved = () => {
  clearGuestInfo()
  setSavedInfo(null)
  reset()
  trackEvent('rsvp_clear_saved_info')
}

const handleClose = () => {
  if (!submitted) trackEvent('rsvp_modal_closed_without_submit', { event_name: cur
  closeModal()
}

if (!isOpen) return null

return (
  <div

```

```

        className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-navy-deep
        onClick={(e) => { if (e.target === e.currentTarget) handleClose() }}
    >
    <div className="relative bg-bg-cream border border-gold w-full max-w-2xl max-h
        <div className="absolute inset-2.5 border border-gold/30 pointer-events-none
        <button
            onClick={handleClose}
            className="absolute top-4 right-4 w-9 h-9 border border-gold text-gold fle
            aria-label="Close"
        >x</button>

    {!submitted ? (
        <>
        <header className="text-center mb-8 pb-6 border-b border-gold/30">
            <div className="font-tenor text-[0.65rem] tracking-[0.5em] text-rose u
            <h3 className="font-italiana text-3xl text-navy-deep mb-2">{currentEve
            <p className="font-playfair italic text-gold">{currentEvent?.location}
        </header>

        {savedInfo && (
            <div className="border border-gold bg-gold/10 p-4 mb-6 text-center for
                Welcome back, <strong className="not-italic text-rose">{savedInfo.na
                <button onClick={handleClearSaved} className="block mx-auto mt-2 for
                Use Different Info
            </button>
        </div>
    )}

    <form onSubmit={handleSubmit(onSubmit)} className="space-y-4">
        /* Form fields – name, email, phone, address, guests, attending */
        /* See full implementation in mockup HTML */
        <button type="submit" disabled={isSubmitting} className="w-full bg-nav
            {isSubmitting ? 'Submitting...' : 'Submit RSVP'}
        </button>
    </form>
</>
) : (
    <div className="text-center py-8">
        <div className="w-16 h-16 mx-auto mb-6 border border-gold rounded-full f
        <h3 className="font-italiana text-3xl text-navy-deep mb-3">Thank You!</h
        <p className="font-playfair italic text-ink mb-2">Your RSVP for</p>
        <div className="inline-block px-6 py-3 bg-bg border border-gold font-ten
            {currentEvent?.name}
        </div>

```

```
        <p className="font-playfair italic text-ink">has been received.</p>
        <p className="font-playfair italic text-gold text-sm mt-4">We'll be in t
        <button onClick={handleClose} className="mt-8 bg-navy-deep text-bg px-10
    </div>
  )}
</div>
</div>
)
```

Note: The full event card markup, gallery grid, and remaining sections follow the same patterns as in `mockup_2_aegean_renaissance_v12.html`. Use that file as the source of truth for exact HTML structure, then port to JSX with Tailwind classes.

8. Amplitude Analytics Integration

8.1 Setup

```
bash

npm install @amplitude/analytics-browser
```

8.2 Initialization

```
tsx
```

```

// lib/amplitude.ts
'use client'
import { useEffect } from 'react'
import * as amplitude from '@amplitude/analytics-browser'

const AMPLITUDE_API_KEY = process.env.NEXT_PUBLIC_AMPLITUDE_API_KEY!

export function AmplitudeProvider({ children }: { children: React.ReactNode }) {
  useEffect(() => {
    amplitude.init(AMPLITUDE_API_KEY, {
      defaultTracking: {
        attribution: true,
        pageViews: true,
        sessions: true,
        formInteractions: false, // we'll do these manually
        fileDownloads: true,
      },
      autocapture: {
        elementInteractions: true, // captures button clicks automatically
      },
    })
  }, [])
  return <{children}</>
}

export function trackEvent(eventName: string, properties?: Record<string, any>) {
  amplitude.track(eventName, {
    ...properties,
    timestamp_iso: new Date().toISOString(),
  })
}

export function identifyUser(email: string, properties?: Record<string, any>) {
  amplitude.setUserId(email)
  if (properties) {
    const identify = new amplitude.Identify()
    Object.entries(properties).forEach(([k, v]) => identify.set(k, v))
    amplitude.identify(identify)
  }
}

```


8.3 Event Taxonomy

Convention: snake_case event names, snake_case property keys. Use semantic, action-based names.

Page-Level Events

- `page_viewed` (auto)
- `session_started` (auto)
- `session_ended` (auto)

Hero Events

- `hero_crest_viewed` — fires on intersection observer when hero loads

Section Visibility (using IntersectionObserver)

- `section_viewed` — properties: `{ section_name: 'countdown' | 'our_story' | 'gallery' | 'events' | 'email_signup' | 'guestbook' | 'footer' }`

Countdown

- `countdown_loaded` — fires once on mount

Gallery

- `gallery_image_clicked` — properties: `{ image_index, image_label, image_type: 'photo' | 'video' }`
- `gallery_video_played` — properties: `{ video_index }`

Events / RSVP Flow

- `event_card_viewed` — properties: `{ event_name }`
- `rsvp_button_clicked` — properties: `{ event_name }`
- `rsvp_modal_opened` — properties: `{ event_name, has_saved_info: boolean }`
- `rsvp_submit_attempted` — properties: `{ event_name, attending, guest_count }`
- `rsvp_submit_success` — properties: `{ event_name, attending }`
- `rsvp_submit_failed` — properties: `{ event_name, error }`
- `rsvp_modal_closed_without_submit` — properties: `{ event_name }`
- `rsvp_clear_saved_info`

Email Signup

- `email_signup_focused`
- `email_signup_submit_attempted` — properties: `{ email_domain }`
- `email_signup_submit_success`
- `email_signup_submit_failed` — properties: `{ error }`

Guestbook

- `guestbook_focused`
- `guestbook_submit_attempted` — properties: `{ message_length }`
- `guestbook_submit_success`
- `guestbook_submit_failed` — properties: `{ error }`

Footer

- `hashtag_clicked` — properties: `{ destination: 'instagram' | 'tiktok' }` (if you make hashtag a link)

8.4 User Identification

When a guest submits an RSVP or email signup, identify them in Amplitude using their email:

```
tsx

import { identifyUser } from '@lib/amplitude'

// On successful RSVP
identifyUser(formData.email, {
  full_name: formData.name,
  guest_count: formData.guests,
  rsvp_status: formData.attending,
  events_rsvped: [currentEvent?.name],
})
```

8.5 IntersectionObserver Pattern for Section Tracking

```
tsx
```

```
// lib/useSectionTracking.ts
'use client'
import { useEffect, useRef } from 'react'
import { trackEvent } from '../amplitude'

export function useSectionTracking(sectionName: string) {
  const ref = useRef<HTMLElement>(null)
  const trackedRef = useRef(false)

  useEffect(() => {
    if (!ref.current) return
    const observer = new IntersectionObserver(
      (entries) => {
        entries.forEach((entry) => {
          if (entry.isIntersecting && !trackedRef.current) {
            trackedRef.current = true
            trackEvent('section_viewed', { section_name: sectionName })
          }
        })
      },
      { threshold: 0.4 }
    )
    observer.observe(ref.current)
    return () => observer.disconnect()
  }, [sectionName])

  return ref
}
```

Use it in each section:

```
tsx

const ref = useSectionTracking('our_story')
return <section ref={ref}>...</section>
```

9. RSVP Backend & Email Capture

9.1 RSVP Endpoint

```
ts
```

```

// app/api/rsvp/route.ts
import { NextResponse } from 'next/server'
import { Resend } from 'resend'

const resend = new Resend(process.env.RESEND_API_KEY)

export async function POST(req: Request) {
  const body = await req.json()

  // 1. Save to Airtable
  const airtableRes = await fetch(
    `https://api.airtable.com/v0/${process.env.AIRTABLE_BASE_ID}/${process.env.AIRTABLE_TABLE_NAME}`,
    {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${process.env.AIRTABLE_API_KEY}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        fields: {
          Event: body.event,
          Name: body.name,
          Email: body.email,
          Phone: body.phone,
          Address: body.address,
          GuestCount: body.guests,
          Attending: body.attending,
          SubmittedAt: new Date().toISOString(),
        },
      }),
    },
  )

  if (!airtableRes.ok) return NextResponse.json({ error: 'Storage failed' }, { status: 500 })

  // 2. Email Merrick & Gillian a notification
  await resend.emails.send({
    from: 'rsvp@happileeeeverafter2027.com',
    to: process.env.NOTIFICATION_EMAIL!,
    subject: `New RSVP: ${body.name} for ${body.event}`,
    html: `
      <h2>New RSVP Received</h2>
      <p><strong>Event:</strong> ${body.event}</p>
    `
  })
}

```

```

    <p><strong>Name:</strong> ${body.name}</p>
    <p><strong>Email:</strong> ${body.email}</p>
    <p><strong>Phone:</strong> ${body.phone}</p>
    <p><strong>Address:</strong> ${body.address}</p>
    <p><strong>Guest Count:</strong> ${body.guests}</p>
    <p><strong>Attending:</strong> ${body.attending}</p>
    `
  },
})

// 3. (Optional) Send confirmation email to the guest
await resend.emails.send({
  from: 'Gillian & Merrick <hello@happileeeeverafter2027.com>',
  to: body.email,
  subject: 'We received your RSVP – thank you!',
  html: `<p>Dear ${body.name.split(' ')[0]}, we've received your RSVP for ${body.e
})

return NextResponse.json({ success: true })
}

```

9.2 Email Subscribe Endpoint

ts

```

// app/api/subscribe/route.ts
import { NextResponse } from 'next/server'

export async function POST(req: Request) {
  const { email } = await req.json()

  await fetch(
    `https://api.airtable.com/v0/${process.env.AIRTABLE_BASE_ID}/${process.env.AIRTABLE_TABLE_NAME}`,
    {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${process.env.AIRTABLE_API_KEY}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        fields: { Email: email, SubscribedAt: new Date().toISOString() },
      }),
    }
  )

  return NextResponse.json({ success: true })
}

```

9.3 Airtable Schema

Base: HappiLee Wedding

Table 1: RSVPs

Field	Type
Event	Single select: "Engagement Soirée", "Bridal Shower", "The Wedding"
Name	Single line text
Email	Email
Phone	Phone number
Address	Long text
GuestCount	Single select: "1", "2", "3", "4", "5", "6+"
Attending	Single select: "yes", "maybe", "no"
SubmittedAt	Date with time

Table 2: EmailSubscribers

Field	Type
Email	Email
SubscribedAt	Date with time

Table 3: Guestbook

Field	Type
Name	Single line text
Message	Long text
SubmittedAt	Date with time
Approved	Checkbox (manual moderation)

9.4 LocalStorage for Persistent Guest Info

```
ts
```

```
// lib/storage.ts
const STORAGE_KEY = 'happilee_guest_info'

export interface GuestInfo {
  name: string
  email: string
  phone: string
  address: string
  guests: string
}

export function saveGuestInfo(info: GuestInfo): void {
  if (typeof window === 'undefined') return
  localStorage.setItem(STORAGE_KEY, JSON.stringify(info))
}

export function loadGuestInfo(): GuestInfo | null {
  if (typeof window === 'undefined') return null
  const data = localStorage.getItem(STORAGE_KEY)
  return data ? JSON.parse(data) : null
}

export function clearGuestInfo(): void {
  if (typeof window === 'undefined') return
  localStorage.removeItem(STORAGE_KEY)
}
```

10. Responsive Behavior

Three breakpoints (mobile-first):

Breakpoint	Tailwind	Width	Notes
Small mobile	(base)	< 480px	Single-column gallery, smaller crest, tightest padding
Mobile	<code>sm:</code>	480px-767px	2-col gallery, stacked event cards, larger crest
Tablet	<code>md:</code>	768px-1023px	2-col grids, condensed sections
Desktop	<code>lg:</code>	1024px+	Full multi-column layouts

Key responsive rules:

- **Hero crest:** 200px → 240px → 260px (responsive scaling)
 - **Hero names:** stack vertically on mobile (each name + ampersand on its own line)
 - **Story:** image-left/text-right desktop → image-top/text-bottom mobile
 - **Gallery:** 4-col → 2-col → 1-col
 - **Events:** 3-column horizontal → fully stacked vertical
 - **Form inputs:** must use `font-size: 16px` on mobile to prevent iOS zoom-on-focus
 - **Touch targets:** all buttons minimum 48px tall on `(hover: none) and (pointer: coarse)` media queries
 - **Hover effects:** disabled on touch devices
-

11. Accessibility Requirements

- **Semantic HTML:** use `<section>`, `<header>`, `<main>`, `<footer>`, `<nav>` correctly
- **Heading hierarchy:** one `<h1>` (hero names), `<h2>` for section titles, `<h3>` for sub-headings
- **Alt text:** all images need descriptive alt text (especially the crest and gallery items)
- **Keyboard navigation:** ensure tab order flows logically; modal must trap focus when open
- **Modal:** trap focus, return focus to trigger button on close, ESC to close (already implemented in mockup), `aria-modal="true"`, `role="dialog"`, `aria-labelledby` pointing to modal title
- **Form labels:** every input needs an associated `<label>`, even if visually hidden
- **Error states:** announce form errors with `aria-live="polite"`
- **Color contrast:** ensure all text passes WCAG AA — note that the gold-on-cream eyebrows are decorative; verify body text passes 4.5:1
- **Reduced motion:** respect `prefers-reduced-motion`:

CSS

```
@media (prefers-reduced-motion: reduce) {  
  *, *::before, *::after { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important; }  
}
```

- **Skip link:** add a "Skip to main content" link at the top of the page for screen readers

12. Performance & SEO

Performance

- **Image optimization:** use Next.js `<Image>` component everywhere; provide `width`, `height`, and `alt`
- **Crest:** convert PNG to WebP for ~50% size reduction; provide `priority` flag in hero
- **Gallery images:** lazy load below-the-fold images, use blur placeholders (`placeholder="blur"`)
- **Videos:** use HLS or MP4 with `poster` images; lazy load and `preload="none"`
- **Fonts:** use `next/font/google` with `display=swap`
- **Bundle size:** target < 150KB JS for initial load; analyze with `@next/bundle-analyzer`
- **Core Web Vitals targets:** LCP < 2.5s, FID < 100ms, CLS < 0.1

SEO

- **Title:** "Gillian & Merrick · #HappiLeeEverAfter2027"
- **Description:** "Sealed in Santorini · Crowned in Jamaica. Join us in celebrating the wedding of Gillian & Merrick Lee, August 2027."
- **OG image:** create a 1200x630 share image featuring the crest + names + tagline
- **Favicon:** use the Lee crest (32x32, 192x192 for PWA)
- **Sitemap:** generate via `next-sitemap`
- **robots.txt:** consider `Disallow: /` to keep the page private (since this is for guests, not public discovery) — discuss with client

13. Deployment Checklist

Pre-launch

- ☐ Set all environment variables in Vercel project
- ☐ Add domain `happileeeeverafter2027.com` in Vercel → Settings → Domains
- ☐ Configure DNS at registrar to point to Vercel (A record / CNAME)
- ☐ Add `www.happileeeeverafter2027.com` redirect to apex
- ☐ Verify SSL certificate provisions automatically
- ☐ Set up Resend domain verification (SPF, DKIM records)

- ☐ Test RSVP flow end-to-end (form submit → Airtable row → email notification)
- ☐ Test email signup flow
- ☐ Verify Amplitude events appear in dashboard within 5 minutes
- ☐ Test on iPhone Safari, Android Chrome, desktop Chrome, Firefox, Safari
- ☐ Run Lighthouse audit; address any score < 90

Launch

- ☐ Push final build to `main` branch
- ☐ Verify production deployment on `happileeeeverafter2027.com`
- ☐ Send test RSVP from incognito window
- ☐ Confirm Amplitude is recording sessions
- ☐ Share live URL with Merrick & Gillian for review

Post-launch

- ☐ Monitor Amplitude weekly for traffic patterns
 - ☐ Monitor Airtable for new RSVPs daily during invitation send periods
 - ☐ Set up Vercel analytics for Core Web Vitals tracking
 - ☐ Schedule a content update sprint as wedding date approaches (real photos, dates, registry, etc.)
-

14. Future Phases

Features intentionally deferred:

1. **Registry section** — code is built but hidden via `display: none`. Re-enable when registries are finalized.
 2. **Live event dates** — currently all show "Date to be announced". Replace with real dates as confirmed.
 3. **Photo gallery upload pipeline** — current gallery uses placeholder mosaic; build admin UI or use Cloudinary for real photo management.
 4. **Travel & lodging info section** — for Jamaica destination wedding (hotels, transportation, dress code, weather)
 5. **Bridal party intro section** — photos and bios of wedding party members
 6. **Multi-language support** — if needed for international guests
 7. **Real-time guestbook display** — currently a write-only form; future iteration could display approved guestbook entries publicly
-

Reference Files

- **Mockup HTML:** `mockup_2_aegean_renaissance_v12.html` — source of truth for design, copy, and structure
- **Crest image:** `Lee_Crest_2026.png`

Contact

- **Client:** Merrick Lee — primary point of contact
- **Domain owner:** Merrick Lee
- **Amplitude API key:** `a013a8aef5661df201bf8330107f5244` (production)

~ Sealed in Santorini · Crowned in Jamaica ~ #HappiLeeEverAfter2027